Algorithmes Usuels

🧱 Calculs sur une liste

Implémentations classiques impératives utilisant des boucles `for`.

root@python:~$

```
# Recherche du maximum
def maximum(liste):
    m = liste[0]
    for i in range(1, len(liste)):
        if liste[i] > m:
            m = liste[i]
    return m

Calcul de la moyenne
```

```
def moyenne(liste):
    somme = 0
    for x in liste:
        somme = somme + x
    return somme / len(liste)
```

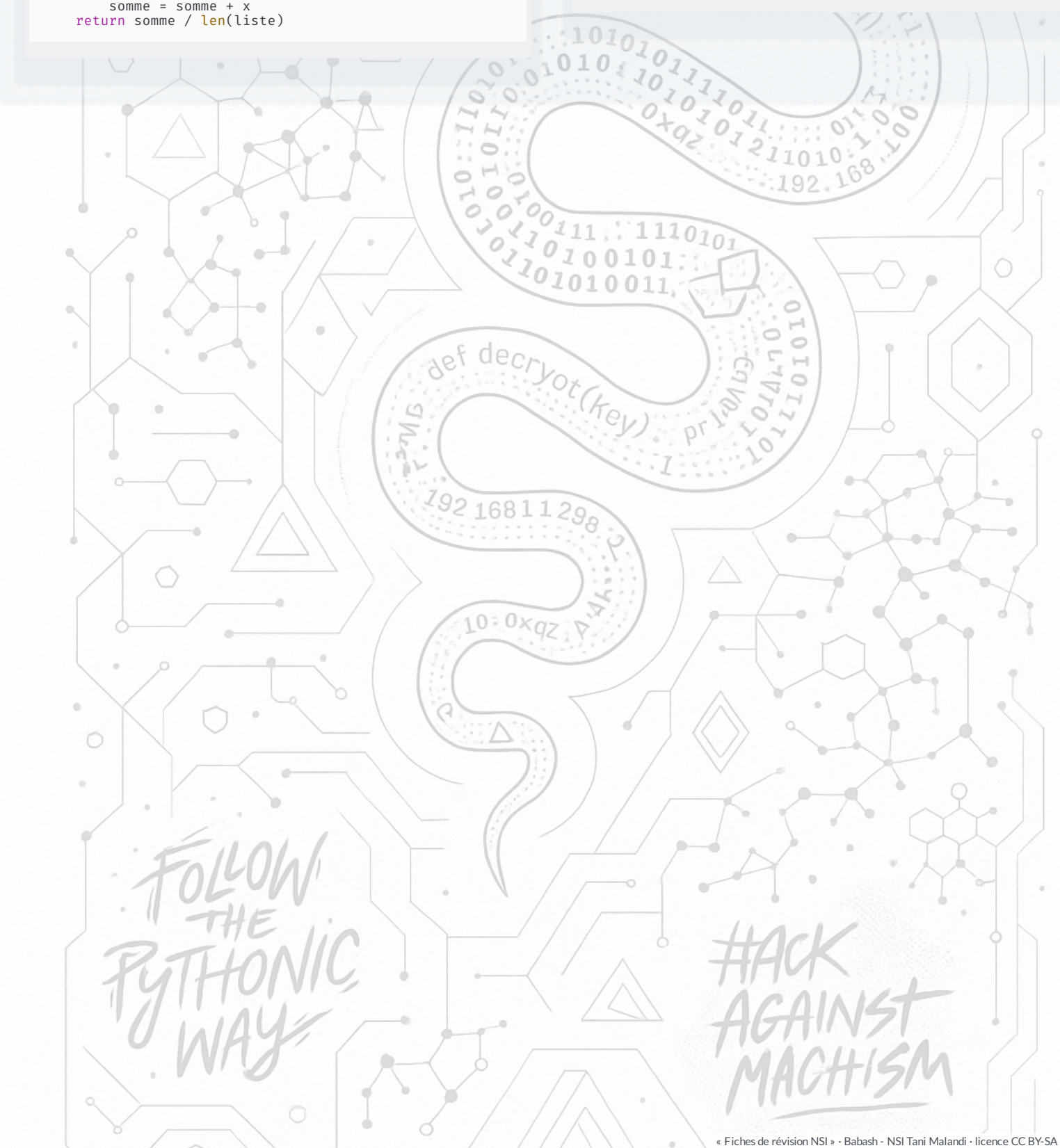
🔍 Recherche et Filtre

root@python:~$

```
# Recherche linéaire
def recherche(liste, cible):
    for i in range(len(liste)):
        if liste[i] == cible:
            return i # Retourne l'index
    return -1 # Non trouvé

Filtrer les éléments (ex: garder les pairs)
```

```
def filtrer_pairs(liste):
    resultat = []
    for x in liste:
        if x % 2 == 0:
            resultat.append(x)
    return resultat
```



« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Le langage SQL

🔍 Consultation (SELECT)

La commande SELECT permet de récupérer des données dans une table. On peut filtrer avec WHERE et trier avec ORDER BY.

```
root@python:~$
```

```sql
-- Récupérer le nom et le prix des produits de plus de 10€
SELECT nom, prix
FROM produits
WHERE prix > 10
ORDER BY prix DESC;
```

```sql
-- Récupérer toutes les colonnes
SELECT * FROM eleves;
```

➕ Action sur les données (CRUD)

Les opérations de mise à jour et d'insertion permettent de manipuler le contenu des tables.

```
root@python:~$
```

```sql
-- Insérer une nouvelle ligne
INSERT INTO eleves (nom, prenom, classe)
VALUES ('Dupont', 'Jean', 'Terminale');

-- Mettre à jour une valeur
UPDATE produits
SET prix = 12.5
WHERE id = 4;
```

```sql
-- Supprimer une ligne
DELETE FROM eleves
WHERE id = 10;
```

📊 Agrégateurs et Jointures

Les fonctions d'agrégation permettent d'effectuer des calculs sur un ensemble de lignes.

```
root@python:~$
```

```sql
SELECT COUNT(*) FROM eleves;          -- Compte le nombre total
SELECT AVG(note) FROM evaluations;  -- Moyenne des notes
SELECT MAX(prix) FROM produits;     -- Prix le plus élevé
```

La **jointure** permet de lier deux tables grâce à une clé étrangère.

```
root@python:~$
```

```sql
SELECT eleves.nom, classes.nom_classe
FROM eleves
JOIN classes ON eleves.classe_id = classes.id;
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Arbres

🌳 Vocabulaire des Arbres

Un arbre est une structure hiérarchique composée de **nœuds**.

- **Racine :** Le nœud supérieur unique.
- **Feuille :** Un nœud sans enfant.
- **Taille :** Nombre total de nœuds.
- **Hauteur :** Longueur du plus long chemin entre la racine et une feuille.

🧭 Parcours d'un arbre

Il existe deux grandes familles de parcours :

1. **Parcours en largeur (BFS) :** On parcourt étage par étage (utilise une **file**).
2. **Parcours en profondeur (DFS) :**
   - **Préfixe :** Racine, Gauche, Droit.
   - **Infixe :** Gauche, Racine, Droit.
   - **Suffixe :** Gauche, Droit, Racine.

root@python:~$

```
def parcours_prefixe(noeud):
    if noeud is not None:
        print(noeud.valeur)
        parcours_prefixe(noeud.gauche)
        parcours_prefixe(noeud.droit)
```

⚙️ Implémentation d'un Arbre Binaire

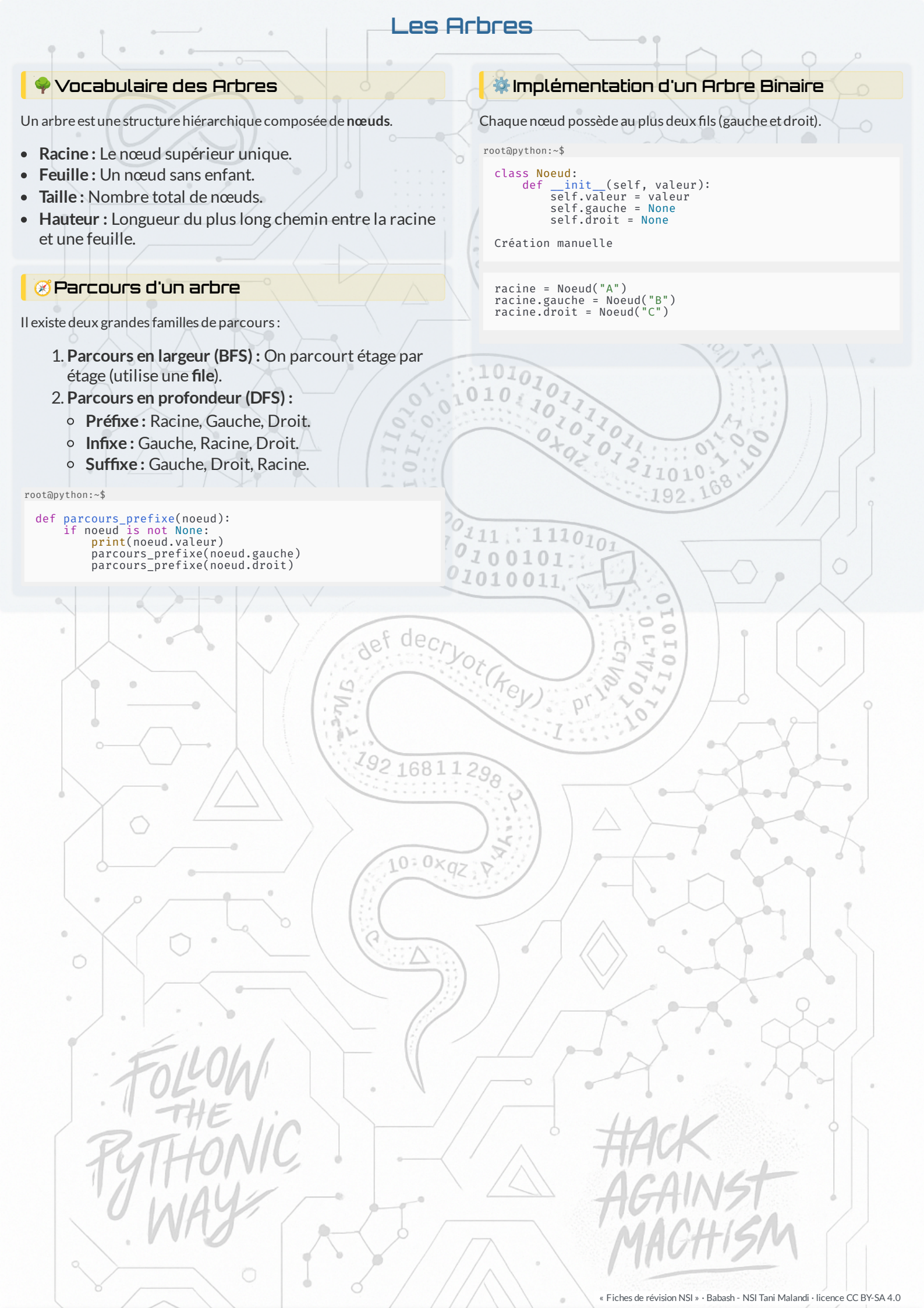
Chaque nœud possède au plus deux fils (gauche et droit).

root@python:~$

```
class Noeud:
    def __init__(self, valeur):
        self.valeur = valeur
        self.gauche = None
        self.droit = None
```

Création manuelle

```
racine = Noeud("A")
racine.gauche = Noeud("B")
racine.droit = Noeud("C")
```



« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Chaînes de Caractères en Python

💡 Définition et Création

Une **chaîne de caractères** (ou `string`) est une séquence ordonnée et **immuable** de caractères. On la reconnaît par les guillemets simples ' ' ou doubles " " qui l'entourent. Une chaîne de caractères peut contenir des lettres, des chiffres, des symboles et des espaces.

```
root@python:~$
```

```
# Création d'une chaîne de caractères vide
ma_chaine = ""
# Création d'une chaîne avec du texte
salut = "Bonjour le monde"
# Obtenir la longueur d'une chaîne avec len()
longueur = len(salut)
print(f"La chaîne 'salut' contient {longueur}
caractères.")
# Affiche "La chaîne 'salut' contient 16 caractères."
```

⚙️ Saisie et Traitement

Saisie avec `input()`

La fonction `input()` permet de demander à l'utilisateur de saisir une chaîne de caractères. Le programme s'arrête en attendant que l'utilisateur entre du texte et appuie sur Entrée. La valeur saisie est renvoyée sous forme de chaîne de caractères.

```
root@python:~$
```

```
nom = input("Entrez votre nom : ")
print(f"Bonjour, {nom} !")
# Si l'utilisateur tape "Alice"
# la console affichera : "Bonjour, Alice !"
```

Méthodes de modification de la casse

Les chaînes de caractères sont **immuables**, ce qui signifie que vous ne pouvez pas modifier un caractère existant directement. Les méthodes suivantes renvoient une **nouvelle** chaîne de caractères modifiée.

.upper() : renvoie une nouvelle chaîne avec tous les caractères en majuscules.

.lower() : renvoie une nouvelle chaîne avec tous les caractères en minuscules.

.capitalize() : renvoie une nouvelle chaîne avec la première lettre en majuscule.

```
root@python:~$
```

```
chaine = "Hello World"
print(chaine.upper())    # Affiche "HELLO WORLD"
print(chaine.lower())    # Affiche "hello world"
```

🤔 Immuabilité

⚠️ Une chaîne de caractères ne peut pas être modifiée après sa création. Toutes les opérations qui semblent "modifier" une chaîne, comme le remplacement d'un caractère, créent en réalité une **nouvelle** chaîne en mémoire.

```
root@python:~$
```

```
slogan = "J'adore Python"
# Cette instruction générera une erreur !
# slogan[8] = 'C'
# TypeError: 'str' object does not support item
assignment
# Solution : créer une nouvelle chaîne
nouveau_slogan = slogan[:8] + "le Python"
print(nouveau_slogan) # Affiche "J'adore le Python"
```

🧭 Indexation et Accès aux éléments

Chaque caractère d'une chaîne possède une position, appelée index. En Python, ⚠️ **l'indexation commence à 0**.

On peut aussi utiliser des index négatifs pour accéder aux caractères à partir de la fin de la chaîne. L'index -1 correspond au dernier caractère, -2 à l'avant-dernier, et ainsi de suite.

```
root@python:~$
```

```
mot = "Python"
# Indexation positive
print(mot[0])   # Affiche "P"
# Indexation négative
print(mot[-1])  # Affiche "n"
# Le slicing : Extraire une partie de la chaîne
[début:fin]
# ⚠️ La fin est exclue !
print(mot[1:4]) # Affiche "yth"
print(mot[2:])  # Affiche "thon"
```

📜 Parcourir une chaîne

La boucle for est l'outil principal pour parcourir une chaîne de caractères.

1. Par caractère :

C'est la méthode la plus simple et la plus courante.

```
root@python:~$
```

```
mot = "code"
for caractere in mot:
  print(caractere)
```

```
Affiche :
c
o
d
e
```

2. Par index :

Utile si vous avez besoin de la position du caractère en plus de sa valeur. On utilise la fonction range(len(chaine)).

```
root@python:~$
```

```
mot = "code"
for i in range(len(mot)):
  print(f"Caractère n°{i} : {mot[i]}")
```

```
Affiche :
Caractère n°0 : c
Caractère n°1 : o
Caractère n°2 : d
Caractère n°3 : e
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Dictionnaires en Python

💡 Définition et Création

Un **dictionnaire** est une collection de données **non ordonnée** et **modifiable**. Contrairement aux listes qui utilisent des index numériques, un dictionnaire stocke des données sous forme de paires **clé-valeur**. Chaque clé est unique et sert d'identifiant pour accéder à sa valeur. Un dictionnaire est reconnaissable par les accolades {} qui l'entourent.

```
root@python:~$
```

```
# Création d'un dictionnaire vide
mon_dictionnaire = {}
# Création d'un dictionnaire avec des éléments
personne = {"nom": "Alice", "age": 25, "ville": "Paris"}
# Obtenir le nombre d'éléments avec len()
nombre_elements = len(personne)
print(f"personne contient {nombre_elements} éléments.")
# Affiche "personne contient 3 éléments."
```

⚙️ Modification du dictionnaire

Ajouter et modifier des éléments

Vous pouvez ajouter une nouvelle paire clé-valeur ou modifier la valeur d'une clé existante en utilisant la syntaxe de l'indexation.

```
root@python:~$
```

```
voiture = {"marque": "Ford", "modele": "Mustang"}
# Ajout d'un nouvel élément
voiture["annee"] = 1964
print(voiture)
# {'marque': 'Ford', 'modele': 'Mustang', 'annee': 1964}
# Modification d'un élément existant
voiture["modele"] = "Focus"
print(voiture)
# {'marque': 'Ford', 'modele': 'Focus', 'annee': 1964}
```

Supprimer des éléments

pop(key) : supprime l'élément correspondant à la clé et renvoie sa valeur.

del : supprime l'élément avec la clé spécifiée.

```
root@python:~$
```

```
livre = {"titre": "1984", "auteur": "George Orwell", "annee":
1949}
# Supprimer un élément avec pop()
annee_supprimee = livre.pop("annee")
print(f"Élément supprimé : {annee_supprimee}")
# Élément supprimé : 1949
print(livre) # {'titre': '1984', 'auteur': 'George Orwell'}
# Supprimer un élément avec del
del livre["auteur"]
print(livre) # {'titre': '1984'}
```

🤔 Copier un dictionnaire

⚠️ Si vous attribuez un dictionnaire à une autre variable avec le signe =, vous ne faites pas une copie ! Les deux variables pointent vers le même objet en mémoire. On appelle cela une référence.

Pour faire une vraie copie, vous devez utiliser la méthode **.copy()**.

```
root@python:~$
```

```
dict_a = {"a": 1, "b": 2}
dict_b = dict_a  # C'est une référence
dict_b["c"] = 3
print(dict_a) # Affiche {'a': 1, 'b': 2, 'c': 3} !
print(dict_b) # Affiche {'a': 1, 'b': 2, 'c': 3}
```

```
dict_c = dict_a.copy()  # Vraie copie avec .copy()
dict_c["d"] = 4
print(dict_a) # Affiche {'a': 1, 'b': 2, 'c': 3}
print(dict_c) # Affiche {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

🧭 Accès aux éléments

Vous accédez à la valeur d'un dictionnaire en utilisant sa clé. ⚠️ Si la clé n'existe pas, une erreur est générée.

```
root@python:~$
```

```
etudiants = {
  "id_1": "Jean",
  "id_2": "Marie",
  "id_3": "Paul"
}
# Accès par la clé
print(etudiants["id_1"])   # Affiche "Jean"
# .get() avec une valeur par défaut
print(etudiants.get("id_2", "Non trouvé"))  # "Marie"
print(etudiants.get("id_4", "Non trouvé"))  # "Non trouvé"
# Utilisation de la méthode .get() sans valeur par défaut
print(etudiants.get("id_2"))  # Affiche "Marie"
#
```

⚠️ Remarque : La méthode .get() ne génère pas d'erreur si la clé n'existe pas, elle renvoie None.⚠️

📜 Parcourir un dictionnaire

La boucle for est l'outil principal pour parcourir un dictionnaire.

1. Par les clés : C'est la méthode la plus simple et la plus courante.

```
root@python:~$
```

```
profil = {"nom": "Alex", "age": 30, "poste": "Designer"}
for cle in profil:
  print(cle)
Affiche :
nom
age
poste
```

2. Par les valeurs : On utilise la méthode **.values()** pour parcourir uniquement les valeurs.

```
root@python:~$
```

```
profil = {"nom": "Alex", "age": 30, "poste": "Designer"}
for valeur in profil.values():
  print(valeur)
Affiche :
Alex
30
Designer
```

3. Par les paires clé-valeur (avec .items()) : La méthode la plus élégante pour avoir à la fois la clé et la valeur.

```
root@python:~$
```

```
profil = {"nom": "Alex", "age": 30, "poste": "Designer"}
for cle, valeur in profil.items():
  print(f"La clé '{cle}' correspond à la valeur '{valeur}'.")
Affiche :
La clé 'nom' correspond à la valeur 'Alex'.
La clé 'age' correspond à la valeur '30'.
La clé 'poste' correspond à la valeur 'Designer'.
```

🕵️ Vérifier la présence d'une clé

L'opérateur **in** est utilisé pour tester si une clé est présente dans un dictionnaire. Il renvoie un booléen : **True** si la clé est trouvée, et **False** sinon.

```
root@python:~$
```

```
eleves = {"prenom": "Léo", "matiere": "Maths", "note": 15}
# Tester si "prenom" est dans le dictionnaire
if "prenom" in eleves:
    print("La clé 'prenom' existe dans le dictionnaire.")
# Affiche "La clé 'prenom' existe dans le dictionnaire."
# Tester si "age" n'est pas dans le dictionnaire
if "age" not in eleves:
    print("La clé 'age' n'est pas présente.")
# Affiche "La clé 'age' n'est pas présente."
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Graphes

🕸️ Vocabulaire et Définitions

Un graphe est composé de **sommets** (ou nœuds) reliés par des **arêtes** (non orienté) ou des **arcs** (orienté).

- **Ordre :** Nombre de sommets.
- **Degré :** Nombre d'arêtes liées à un sommet.
- **Cycle :** Chemin qui revient à son point de départ.

🧭 Algorithmes de Parcours

- **Parcours en Largeur (BFS) :** Utilise une **file**. Trouve le chemin le plus court (en nombre d'arêtes).
- **Parcours en Profondeur (DFS) :** Utilise la **récursivité** (ou une pile). Explore un chemin le plus loin possible avant de revenir.

⚙️ Implémentations (Représentations)

Il existe deux manières classiques de représenter un graphe en mémoire :

1. Matrice d'adjacence : Un tableau 2D (0 ou 1 si lien).

```
root@python:~$
```

```
# Graphe avec 3 sommets A(0), B(1), C(2)
matrice = [
    [0, 1, 1], # A est lié à B et C
    [1, 0, 0], # B est lié à A
    [1, 0, 0]  # C est lié à A
]
```

2. Dictionnaire d'adjacence : Clés = sommets, Valeurs = listes des voisins.

```
root@python:~$
```

```
graphe = {
    'A': ['B', 'C'],
    'B': ['A'],
    'C': ['A']
}
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Listes en Python

💡 Définition et Création

Une liste est une collection de données ordonnée et modifiable, on parle de **Mutable**. Elle est reconnaissable par les crochets **[]** qui l'entourent. On peut y stocker des éléments de types différents (nombres, chaînes de caractères, booléens, etc.).

```
root@python:~$
```

```
# Création d'une liste vide
ma_liste = []
# Création d'une liste avec des éléments
fruits = ["pomme", "banane", "orange"]
# Obtenir la longueur d'une liste avec len()
longueur = len(fruits)
print(f"La liste 'fruits' contient {longueur}
éléments.")
# Affiche "La liste 'fruits' contient 3 éléments."
```

⚙️ Modification de la liste

Ajouter des éléments

append(element) : ajoute un élément à la fin de la liste.
insert(index, element) : insère un élément à un index précis.
extend(iterable) : ajoute tous les éléments d'un autre objet (comme une autre liste) à la fin.

```
root@python:~$
```

```
jours = ["lundi", "mardi"]
jours.append("jeudi")
print(jours) # ['lundi', 'mardi', 'jeudi']
jours.insert(2, "mercredi")
print(jours) # ['lundi', 'mardi', 'mercredi', 'jeudi']
week_end = ["vendredi", "samedi"]
jours.extend(week_end)
print(jours)
# ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi',
'samedi']
```

Supprimer des éléments

pop(index) : supprime l'élément à l'index donné et le renvoie. Si l'index est omis, il supprime et renvoie le dernier élément.

```
root@python:~$
```

```
nombres = [10, 20, 30, 40]
# Supprimer le dernier élément et le stocker
element_supprime = nombres.pop()
print(f"Élément supprimé : {element_supprime}")
# Élément supprimé : 40
print(nombres)    # [10, 20, 30]
```

🤔 Copier une liste

⚠️ Si vous attribuez une liste à une autre variable avec le signe =, vous ne faites pas une copie ! Les deux variables pointent vers le même objet en mémoire. On appelle cela une référence.

Pour faire une vraie copie, on peut utiliser la technique du slicing ou la méthode **.copy()**.

```
root@python:~$
```

```
liste_a = [1, 2, 3]
liste_b = liste_a  # C'est une référence, pas une copie
!
liste_b.append(4)
print(liste_a) # Affiche [1, 2, 3, 4] !
liste_c = liste_a[:] # Vraie copie avec slicing
liste_c.append(5)
print(liste_a) # Affiche [1, 2, 3, 4]
print(liste_c) # Affiche [1, 2, 3, 4, 5]
```

🧭 Indexation et Accès aux éléments

Chaque élément d'une liste possède une position, appelée index. En Python, ⚠️ **l'indexation commence à 0**.

On peut aussi utiliser des index négatifs pour accéder aux éléments à partir de la fin de la liste. L'index -1 correspond au dernier élément, -2 à l'avant-dernier, et ainsi de suite.

```
root@python:~$
```

```
animaux = ["chien", "chat", "oiseau", "poisson"]
# Indexation positive
print(animaux[0])   # Affiche "chien"
# Indexation négative
print(animaux[-1])  # Affiche "poisson"
# Le slicing : Extraire une partie de la liste
[début:fin]
# ⚠️ La fin est exclue !
print(animaux[1:3]) # Affiche ['chat', 'oiseau']
print(animaux[2:])  # Affiche ['oiseau', 'poisson']
```

📜 Parcourir une liste

La boucle for est l'outil principal pour parcourir une liste.

1. Par élément :

C'est la méthode la plus simple et la plus courante. On parcourt directement chaque valeur de la liste.

```
root@python:~$
```

```
pizzas = ["reine", "calzone"]
for pizza in pizzas:
  print(pizza)
```

```
Affiche :
reine
calzone
```

2. Par index :

Utile si vous avez besoin de la position de l'élément en plus de sa valeur. On utilise la fonction range(len(liste)).

```
root@python:~$
```

```
pizzas = ["reine", "calzone"]
for i in range(len(pizzas)):
  print(f"Pizza n°{i} : {pizzas[i]}")
```

```
Affiche :
Pizza n°0 : reine
Pizza n°1 : calzone
```

3. Parcours mixte (avec enumerate) :

La méthode la plus élégante pour avoir à la fois l'index et l'élément. Elle est à privilégier par rapport à la méthode précédente.

```
root@python:~$
```

```
pizzas = ["reine", "calzone"]
for index, pizza in enumerate(pizzas):
  print(f"Index {index} correspond à la {pizza}.")
```

```
Affiche :
Index 0 correspond à la reine.
Index 1 correspond à la calzone.
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les structures de contrôle élémentaires en Python

🚦 Les Conditions (Structures Conditionnelles)

Les conditions permettent d'exécuter un code **uniquement si une expression est vraie**.

Structure `if` (Si)

Exécute un bloc de code si la condition est vraie.

```
root@python:~$
nombre = 10
if nombre > 5:
    print("Le nombre est supérieur à 5.")
# Affiche : Le nombre est supérieur à 5.
```

Explication : L'instruction `print` est exécutée car la condition `nombre > 5` est vraie.

Structure `if...else` (Si...Sinon)

Exécute un premier bloc de code si la condition est vraie, **sinon** exécute un second bloc de code.

```
root@python:~$
age = 15
if age ⩾ 18:
    print("Vous êtes majeur.")
else:
    print("Vous êtes mineur.")
# Affiche : Vous êtes mineur.
```

Explication : La condition `age ⩾ 18` est fausse, donc le code sous le `else` est exécuté.

Structure `if...elif...else` (Si...Sinon Si...Sinon)

Permet de tester **plusieurs conditions** en séquence.

```
root@python:~$
note = 12
if note ⩾ 15:
    print("Très bien")
elif note ⩾ 10:
    print("Bien")
else:
    print("Insuffisant")
# Affiche : Bien
```

Explication : La première condition (`⩾ 15`) est fausse. La deuxième (`elif note ⩾ 10`) est vraie, donc le code sous le `elif` est exécuté et le reste est ignoré.

```
root@python:~$
points = 80
if points ⩾ 50:
    print("A. Vous avez la moyenne.")
elif points ⩾ 75:
    print("B. Vous avez un bon score.")

if points > 70:
    print("C. Votre score dépasse 70.")

# Résultat pour points = 80 :
# A. Vous avez la moyenne.
# C. Votre score dépasse 70.
```

Explication :

->**Ligne A (Active) :** Le programme teste `if points ⩾ 50` (80 >= 50) est **VRAI**. Le message A est affiché. Le bloc if/elif s'arrête ici.
->**Ligne B (Ignorée) :** Le test `elif points ⩾ 75` est Vrai **mais n'est jamais atteint** car le bloc if/elif s'est terminé à la Ligne A.
->**Ligne C (Active) :** Le second `if points > 70` est un **nouveau test indépendant**. Il est VRAI et le message C est affiché.

🔁 Les Boucles (Itérations)

Les boucles permettent de répéter un bloc d'instructions. C'est essentiel pour automatiser des tâches ou parcourir des collections de données.

La Boucle `for` (Itération Définie)

Elle est utilisée lorsque l'on connaît le nombre d'itérations à l'avance, souvent pour parcourir les éléments d'une séquence (comme une liste) ou un intervalle de nombres grâce à la fonction `range( )`.

```
root@python:~$
# Utilisation de range(n) pour répéter n fois (de 0 à n-1)
for i in range(3):
    print(f"Tour numéro {i}") # print permet d'afficher
# Affiche :
# Tour numéro 0
# Tour numéro 1
# Tour numéro 2
```

Explication : `range(3)` génère la séquence 0, 1, 2. La boucle s'exécute 3 fois.

La Boucle `while` (Itération Conditionnelle)

Elle s'exécute **tant qu'une condition reste vraie**. Il faut s'assurer que la condition devienne fausse à un moment pour éviter une **boucle infinie**.

```
root@python:~$
compteur = 0
while compteur < 3:
    print(f"Compteur : {compteur}")
    # Incrémentation pour éviter la boucle infinie
    compteur = compteur + 1
# Affiche :
# Compteur : 0
# Compteur : 1
# Compteur : 2
```

Explication : Le code dans le `while` s'exécute tant que la variable `compteur` est strictement inférieure à 3. À chaque tour, on augmente `compteur` de 1, ce qui permet à la boucle de se terminer.

💡 Opérateurs Logiques (not, and, or)

Ils permettent de combiner ou d'inverser des conditions pour des tests plus complexes.

Opérateur	Signification	Quand est-ce Vrai ?
and	**ET** logique	Si **toutes** les conditions sont Vraies.
or	**OU** logique	Si **au moins une** des conditions est Vraie.
not	**NON** logique	Inverse la condition (Vrai devient Faux, et Faux devient Vrai).

```
root@python:~$
# Exemple avec AND
temperature = 25
ensoleille = True
if temperature > 20 and ensoleille:
    print("Journée idéale pour une sortie.")
# Affiche : Journée idéale pour une sortie.

# Exemple avec OR
heure = 8
jour = "dimanche"
if heure < 9 or jour == "dimanche":
    print("C'est encore l'heure de dormir.")
# Affiche : C'est encore l'heure de dormir.

# Exemple avec NOT
est_fini = False
if not est_fini :
    print("Le travail continue.")
# Affiche : Le travail continue.
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Tuples en Python

💡 Définition et Création

Un **tuple** (ou p-uplet) est une collection de données **ordonnée** mais **immuable** (non modifiable). Une fois créé, on ne peut ni ajouter, ni supprimer, ni modifier ses éléments. Il est reconnaissable par les parenthèses **()** qui l'entourent.

```
root@python:~$
```

```
# Création d'un tuple vide
mon_tuple = ()
# Création d'un tuple avec des éléments
point = (10, 20)
# Un tuple peut contenir des types différents
personne = ("Bob", 18, True)
# Obtenir le nombre d'éléments avec len()
taille = len(personne)
print(f"Le tuple contient {taille} éléments.")
# Affiche "Le tuple contient 3 éléments."
```

⚠️ **Attention :** Pour créer un tuple avec **un seul** élément, il faut obligatoirement mettre une virgule : `mon_tuple = (5,)`

📜 Parcourir un tuple

On utilise la boucle `for` exactement comme avec une liste.

1. Par élément : C'est la méthode la plus simple et la plus courante. On parcourt directement chaque valeur du tuple.

```
root@python:~$
```

```
notes = (12, 15, 18)
for n in notes:
    print(n)
```

```
Affiche :
12
15
18
```

2. Par index : Utile si vous avez besoin de la position de l'élément en plus de sa valeur. On utilise la fonction range(len(t)).

```
root@python:~$
```

```
notes = (12, 15, 18)
for i in range(len(notes)):
    print(f"Note n°{i} : {notes[i]}")
```

```
Affiche :
Note n°0 : 12
Note n°1 : 15
Note n°2 : 18
```

3. Parcours mixte (avec enumerate) : La méthode la plus élégante pour avoir à la fois l'index et l'élément. Elle est à privilégier par rapport à la méthode précédente.

```
root@python:~$
```

```
notes = (12, 15, 18)
for i, n in enumerate(notes):
    print(f"Note n°{i} : {n}")
```

```
Affiche :
Note n°0 : 12
Note n°1 : 15
Note n°2 : 18
```

🧭 Indexation et Accès aux éléments

Comme pour les listes et les chaînes, chaque élément possède un index. ⚠️**L'indexation commence à 0.**

```
root@python:~$
```

```
couleurs = ("rouge", "vert", "bleu", "jaune")
# Accès par index positif
print(couleurs[0])   # Affiche "rouge"
# Accès par index négatif
print(couleurs[-1])  # Affiche "jaune"
# Le slicing : Extraire une partie [début:fin]
print(couleurs[1:3]) # Affiche ('vert', 'bleu')
```

🤔 Immuabilité

⚠️ Contrairement aux listes, le tuple est **immuable**. On ne peut pas modifier son contenu après sa création. C'est une structure sécurisée pour des données qui ne doivent pas changer (ex: coordonnées GPS, constantes).

🧲‼️ On ne peut pas modifier un tuple ‼️🧲

```
root@python:~$
```

```
triplet = (1, 2, 3)
# Cette instruction générera une erreur !
# triplet[0] = 10
# TypeError: 'tuple' object does not support item
assignment
```

🕵️ Vérifier la présence d'une valeur

On utilise l'opérateur **in** pour tester si un élément appartient au tuple.

```
root@python:~$
```

```
langages = ("Python", "C", "Java")
if "Python" in langages:
    print("Le langage est présent.")
# Affiche "Le langage est présent."
if "HTML" not in langages:
    print("Le langage n'est pas présent.")
# Affiche "Le langage n'est pas présent."
if "Python" not in langages:
    print("Le langage est présent.")
# Ne fait rien
if "HTML" in langages:
    print("Le langage n'est pas présent.")
# Ne fait rien
```

⚙️ Affectation Multiple (Unpacking)

Le tuple permet d'affecter plusieurs variables d'un coup. C'est une fonctionnalité très utilisée en Python, notamment pour retourner plusieurs valeurs avec une fonction.

1. Affectation multiple

```
root@python:~$
```

```
coordonnees = (48.85, 2.35)
latitude, longitude = coordonnees
print(latitude)  # 48.85
```

2. Échanger deux variables sans variable temporaire

```
root@python:~$
```

```
a = 5
b = 10
a, b = b, a
print(a, b) # Affiche 10 5
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Les Variables et Opérateurs en Python

💡 Définition et Création

Une **variable** est un espace de stockage nommé qui contient une valeur. En Python, vous n'avez pas besoin de déclarer le type de la variable ; l'interpréteur le détermine automatiquement en fonction de la valeur que vous lui attribuez.

💾 Types de Données de Base

Python dispose de plusieurs types de données fondamentaux :

- **Nombres entiers (`int`)** : Pour les nombres sans décimales. Exemple : `age = 25`.
- **Nombres flottants (`float`)** : Pour les nombres à virgule. Exemple : `taille = 1.75`.
- **Chaînes de caractères (`str`)** : Pour le texte. Elles sont délimitées par des guillemets simples ou doubles. Exemple : `nom = "Alice"`.
- **Booléens (`bool`)** : Représentent une valeur de vérité, qui peut être soit **True** (vrai) soit **False** (faux). Exemple : `est_majeur = True`.

📝 Règles pour nommer les variables

Les noms de variables doivent suivre des règles précises pour être valides en Python. S'ils ne les respectent pas, cela entraînera une erreur de syntaxe.

- Un nom de variable doit commencer par une lettre (`a-z`, `A-Z`) ou un trait de soulignement (`_`). Il ne peut pas commencer par un chiffre.
- Un nom de variable ne peut contenir que des caractères alphanumériques (`a-z`, `A-Z`, `0-9`) et des traits de soulignement (`_`).
- Les noms de variables sont sensibles à la casse (`age`, `Age` et `AGE` sont des variables différentes).
- Un nom de variable ne peut pas être un mot-clé Python réservé (comme `if`, `for`, `class`, etc.).

```
root@python:~$
```

```python
# Exemples de noms de variables valides
mon_age = 30
_nom_utilisateur = "admin"
vitesse_1 = 120
```

🛠️ Les Opérateurs Fondamentaux

1. L'Opérateur d'Affectation (=)

Cet opérateur est utilisé pour attribuer une valeur à une variable. Les opérateurs combinés permettent de simplifier les opérations d'affectation en les fusionnant avec un opérateur arithmétique.

```
root@python:~$
```

```python
# Affectation d'une valeur à une variable
a = 10
# Opérateur d'affectation combiné
a += 5  # équivaut à a = a + 5. 'a' vaut maintenant 15.
a -= 2  # équivaut à a = a - 2. 'a' vaut maintenant 13.
a *= 3  # équivaut à a = a * 3. 'a' vaut maintenant 39.
a /= 2  # équivaut à a = a / 2. 'a' vaut maintenant 19.5.
```

2. Les Opérateurs Arithmétiques ➗

Ils servent à effectuer des opérations mathématiques.

Opérateur	Description	Exemple
`+`	Addition	`5 + 3` donne `8`
`-`	Soustraction	`10 - 4` donne `6`
`*`	Multiplication	`2 * 5` donne `10`
`/`	Division	`10 / 3` donne `3.333...`
`//`	Division entière	`10 // 3` donne `3`
`%`	Modulo (reste de la division)	`10 % 3` donne `1`
`**`	Puissance	`2 ** 3` donne `8`

3. Les Opérateurs de Comparaison ⚖️

Ils comparent deux valeurs et renvoient toujours un résultat booléen (**True** ou **False**).

Opérateur	Description	Exemple
$=$	Égal à	$5 = 5$ est **True**
$\neq$	Différent de	$5 \neq 6$ est **True**
$>$	Strictement supérieur à	$5 > 3$ est **True**
$<$	Strictement inférieur à	$5 < 3$ est **False**
$\geqslant$	Supérieur ou égal à	$5 \geqslant 5$ est **True**
$\leqslant$	Inférieur ou égal à	$5 \leqslant 3$ est **False**

4. Les Opérateurs Logiques 🧠

Ils permettent de combiner des conditions booléennes.

- **`and`** : L'expression est **True** si **toutes** les conditions sont **True**.
- **`or`** : L'expression est **True** si au moins **une** des conditions est **True**.
- **`not`** : Inverse le résultat de la condition.

```
root@python:~$
```

```python
x = 5
y = 10
print(x > 0 and y > 0) # Affiche True
print(x > 10 or y > 5)  # Affiche True
print(not (x == 5))     # Affiche False
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Piles et Files

📚 La Pile (Stack - LIFO)

Le dernier élément ajouté est le premier à sortir (**Last In, First Out**).
Analogie : une pile d'assiettes.

- **Empiler (push) :** Ajouter au sommet.
- **Dépiler (pop) :** Retirer le sommet.

```
root@python:~$
```

```
# Implémentation avec une liste
pile = []
pile.append(10)  # Empiler
pile.append(20)
sommet = pile.pop() # Dépiler → renvoie 20
```

⏳ La File (Queue - FIFO)

Le premier élément ajouté est le premier à sortir (**First In, First Out**).
Analogie : une file d'attente au guichet.

- **Enfiler (enqueue) :** Ajouter à la fin.
- **Défiler (dequeue) :** Retirer du début.

```
root@python:~$
```

```
# Implémentation avec une liste
file = []
file.append("Client 1") # Enfiler
file.append("Client 2")
prochain = file.pop(0)  # Défiler → renvoie "Client 1"
```

⚙️ Opérations usuelles

Quelle que soit la structure, on doit pouvoir tester si elle est vide et connaître sa taille.

```
root@python:~$
```

```
def est_vide(s):
    return len(s) == 0
```

```
def taille(s):
    return len(s)
```

« Fiches de révision NSI » · Babash - NSI Tani Malandi · licence CC BY-SA 4.0

Programmation Orientée Objet (POO)

💡 Concept et Définition

La **POO** (Programmation Orientée Objet) est un **paradigme de programmation** qui consiste à définir des "objets" informatiques. Au lieu de voir un programme comme une simple suite d'instructions, on le conçoit comme une interaction entre des entités autonomes.

L'analogie du Robot :

->**La Classe (Le Plan) :** C'est le schéma technique à l'usine. Il définit que tout robot *doit* avoir un nom et une batterie, et *sait* marcher. Ce n'est pas encore un robot réel, c'est un **nouveau type** de donnée que vous créez.

->**L'Objet ou Instance (Le Robot réel) :** C'est l'unité qui sort de l'usine. On peut créer des "jumeaux" issus du même plan : deux robots identiques au départ, mais qui mèneront leur propre vie.

->**L'Attribut (Caractéristique) :** Ce sont les propriétés du robot (son nom, son niveau de batterie). Même s'ils ont le même constructeur, un robot peut être à 100% de batterie et l'autre à 10%.

->**La Méthode (Action) :** Ce sont les capacités du robot (marcher, saluer). C'est une fonction interne à l'objet.

->**Le Constructeur (L'assemblage) :** C'est l'étape de fabrication qui donne ses caractéristiques initiales au robot au moment où il "naît".

📊 Les Attributs : Accès et Modification

Un **attribut** est une variable interne à l'objet qui définit son état. On utilise la notation pointée `.` pour agir dessus. On trouve usuellement les attributs dans le constructeur (__init__).

Accès :

```
root@python:~$
# Accès à un attribut
print(mon_robot.nom)  # Affiche "R2D2"
```

Modification :

```
root@python:~$
# Modification d'un attribut
mon_robot.v = 2.0
print(mon_robot.v) # Affiche 2.0
```

⚙️ Les Méthodes et leurs Appels

Une **méthode** est une fonction définie à l'intérieur d'une classe. Elle représente un comportement de l'objet. Le paramètre `self` permet à la méthode d'accéder aux attributs de l'objet qui l'appelle.

Définition de méthode :

```
root@python:~$
class Robot:
    def __init__(self, nom, version=1.0):
        # Initialisation des attributs (état de l'objet)
        self.nom = nom          # Passé à l'assemblage
        self.v = version        # Valeur par défaut
        self.batt = 100         # Valeur fixe au départ

    def saluer(self): # Définition d'une méthode
        print(f"Bonjour, je suis {self.nom}")

    def charger(self, energie):
        self.batt += energie
```

Appel de méthode :

Voyons maintenant un exemple d'appel de la méthode.

```
root@python:~$
# Appel de méthode
mon_robot.saluer()      # Affiche "Bonjour, je suis R2D2"
mon_robot.charger(20)   # Modifie l'état interne (batterie)
```

🏗️ Le Constructeur

En Python, le constructeur s'appelle `__init__`, il permet de donner une valeur aux attributs d'un objet. Il est appelé à la création de chaque objet. C'est ici que l'on transforme le plan en un objet concret avec ses propres valeurs.

```
root@python:~$
class Robot:
    def __init__(self, nom, version=1.0):
        # Initialisation des attributs (état de l'objet)
        self.nom = nom          # Passé à l'assemblage
        self.v = version        # Valeur par défaut
        self.batt = 100         # Valeur fixe au départ
```

Contrairement aux autres méthodes, on ne l'appelle jamais manuellement (on n'écrit pas `robot.__init__()`) : Python l'exécute **automatiquement** en arrière-plan au moment précis où l'on crée l'objet (en utilisant le nom de la classe `Robot()`) pour garantir qu'il naisse dans un état valide.

🏗️ Création d'objet (Instanciation)

Pour créer un objet on utilise **le nom de la classe** comme si c'était une fonction. La fonction véritablement appelée est `__init__` mais on ne l'appelle jamais explicitement.

Création d'objet :

Création de "jumeaux" (Appel automatique de __init__).

```
root@python:~$
robot_A = Robot("R2D2")
robot_B = Robot("R2D2")
robot_C = Robot("C3PO", 4.0)
```

Indépendance des objets :

Ils sont du même type (Robot) mais sont des objets distincts.

```
root@python:~$
robot_A.batt = 20  # Robot_A est déchargé, mais pas Robot_B !
print(robot_A.batt) # Affiche 20
print(robot_B.batt) # Affiche 100
```

🆔 Comprendre le mot-clé "self"

Self représente l'instance elle-même (le "moi"). C'est le lien entre le code générique de la classe et l'objet spécifique en mémoire.

->**Pourquoi est-il partout ?** Dans la classe, Python ne sait pas encore quel robot va appeler la méthode. `self` dit : "Applique cette action sur l'objet qui m'a appelé".

->**Le paramètre invisible :** Il doit être le **premier argument** de chaque méthode dans la classe, mais on ne lui passe **jamais** de valeur lors de l'appel.

```
root@python:~$
def afficher_nom(self):
    # Sans "self.", Python chercherait une variable locale
    # Avec "self.", il va chercher l'attribut DANS l'objet.
    print(f"Mon nom est {self.nom}")
```

Self n'est jamais utilisé en dehors de la classe (en dehors des méthodes).

```
root@python:~$
robot_A.afficher_nom()
# Python fait secrètement : afficher_nom(robot_A)
```